



PRISQ Progressive Web App (PWA) – Architecture & Implementation

Overall architecture of Quiz

The PRISQ Quiz is structured into:

- Configuration layer
- Data layer
- Rendering engine
- Scoring engine
- Result display system

It dynamically loads questions, renders UI based on type, and calculates risk using either a point-based or logistic regression model.

Scoring Model Configuration

Defines which scoring engine is active.

```
const PRIMARY_MODEL = "beta";
```

What It Does

- "quantized" → Uses point-based scoring
- "beta" → Uses logistic regression probability model
-

🔧 How to Modify

- Switch to quantized scoring:

```
const PRIMARY_MODEL = "quantized";
```

- Add a new model → Extend logic in `showResults()` and scoring functions.

Runtime State Management

Tracks quiz progress and user answers during session.

```
let currentQuestionId = null;  
let answers = {};  
let answerHistory = [];
```

What It Does

- currentQuestionId → active question
- answers → stores user selections
- answerHistory → enables “Previous” navigation

How to Modify

- Persist answers in localStorage for session recovery.
- Add analytics tracking when answers change.

Data Loading & Initialization

```
async function initQuiz() {  
  quizSourceData = await fetch("data/quiz_data.json")  
    .then(r => r.json());  
  quizData = buildQuizData(quizSourceData, getCurrentLanguage());  
  resetQuiz();  
  loadQuestion();  
}
```

What It Does

1. Loads JSON configuration
2. Applies language localization
3. Initializes state
4. Renders first question

How to Modify

- Change data source to API endpoint
- Add loading spinner before rendering

Localization System

Supports Arabic and English dynamically.

```
function getCurrentLanguage() {  
  return window.PrisqSite?.getCurrentLanguage?.() === "ar" ? "ar"  
  : "en";  
}
```

What It Does

- Detects site language
- Rebuilds quiz text dynamically
- Allows mid-quiz language switching

How to Modify

- Add new language by extending JSON data
- Improve fallback handling

Question Rendering Engine

Determines how each question is displayed.

```
if (renderQuestion.inputType === "number_select") {  
  renderNumberSelect(renderQuestion);  
} else if (renderQuestion.inputType === "number_input") {  
  renderNumberInput(renderQuestion);  
} else {  
  renderOptions(renderQuestion);  
}
```

What It Does

Supports 3 UI types:

- Multiple-choice buttons
- Dropdown numeric selector
- Manual numeric input

How to Modify

- Add new input type (e.g., slider)
- Add animations between transitions

Answer Handling

Stores selected answer and moves to next question.

```
function selectAnswer(question, answer) {  
  answers[question.id] = answer;  
  answerHistory.push(question.id);  
  currentQuestionId = answer.next || question.next;  
  loadQuestion();  
}
```

What It Does

Saves answer

Maintains history

Determines next question dynamically

How to Modify

Quantized Scoring Model

Calculates risk using point-based scoring.

```
let score = 0;
Object.values(answers).forEach(answer => {
  if (answer.scoreValue) {
    score += answer.scoreValue;
  }
});
```

What It Does

- Adds answer scores
- Applies BMI bands
- Applies blood pressure bands
- Maps total score → risk category

How to Modify

- Adjust scoring weights in JSON
- Add new scoring dimension

Logistic Regression (Beta Model)

Calculates probability using logistic regression.

```
const z = intercept +  
    (betas.age * age) +  
    (betas.bmi * bmi);
```

```
const probability = 1 / (1 + Math.exp(-z));
```

What It Does

- Computes linear combination of health metrics
- Converts to probability
- Maps probability → risk category

How to Modify

- Adjust beta coefficients in JSON
- Add additional predictors (e.g., smoking status)

Result Rendering

Displays final risk result.

```
if (PRIMARY_MODEL === "quantized") {  
    finalScoreElement.textContent = String(quantizedScore);  
} else {  
    finalScoreElement.textContent =  
        `${(betaResult.probability * 100).toFixed(1)}%`;  
}
```

What It Does

- Selects active model
- Displays score or probability
- Updates risk label styling
- Shows encouragement message

Role of site.js in the Architecture

site.js controls the main website behavior around the quiz.

It handles:

- Language switching
- Page direction (LTR / RTL)
- UI text updates
- Basic navigation
- Global utilities

Website Layer

- site.js (controller)
- quiz.js (logic engine)
- quiz_data.json (data layer)

Global Namespace Pattern

```
window.Pris  
window.PrisqSite = {  
  getLanguage,  
  setLanguage,  
  updateUI  
};
```

What This Means

- Creates one global object
- Other files (like quiz.js) can use it
- Keeps functions organized

Language Storage

```
let currentLanguage =  
  localStorage.getItem("prisq_lang") || "en";
```

What This Does

- Loads saved language from browser storage
- Defaults to English if nothing saved
- Keeps language preference after refresh
- can add new language

Changing Language

```
function setLanguage(lang) {  
  currentLanguage = lang;  
  localStorage.setItem("prisq_lang", lang);  
  updateUI();  
}
```

What This Does

1. Updates language variable
2. Saves it in browser
3. Refreshes the UI text

Button & Navigation Control

```
document.querySelector("#startBtn")  
  .addEventListener("click", () => {  
    document.querySelector("#quizSection")  
      .scrollIntoView({ behavior: "smooth" });  
  });
```

What This Does

- Listens for button click
- Scrolls to quiz section smoothly
- Improves user experience